HIGH PERFORMANCE COMPUTING FOR SCIENCE AND ENGINEERING

Harvard University
CS 2050

Lecture 16

March 26, 2026

Spring Recess

Week 8 - Extended Example: Training Large Language Models

Week 9 - Extended Example: Distributed Linear Algebra

Week 10 - Extended Example: Molecular Dynamics

Week 11 - Extra Topic: Kokkos

Week 12 - Project Work Week

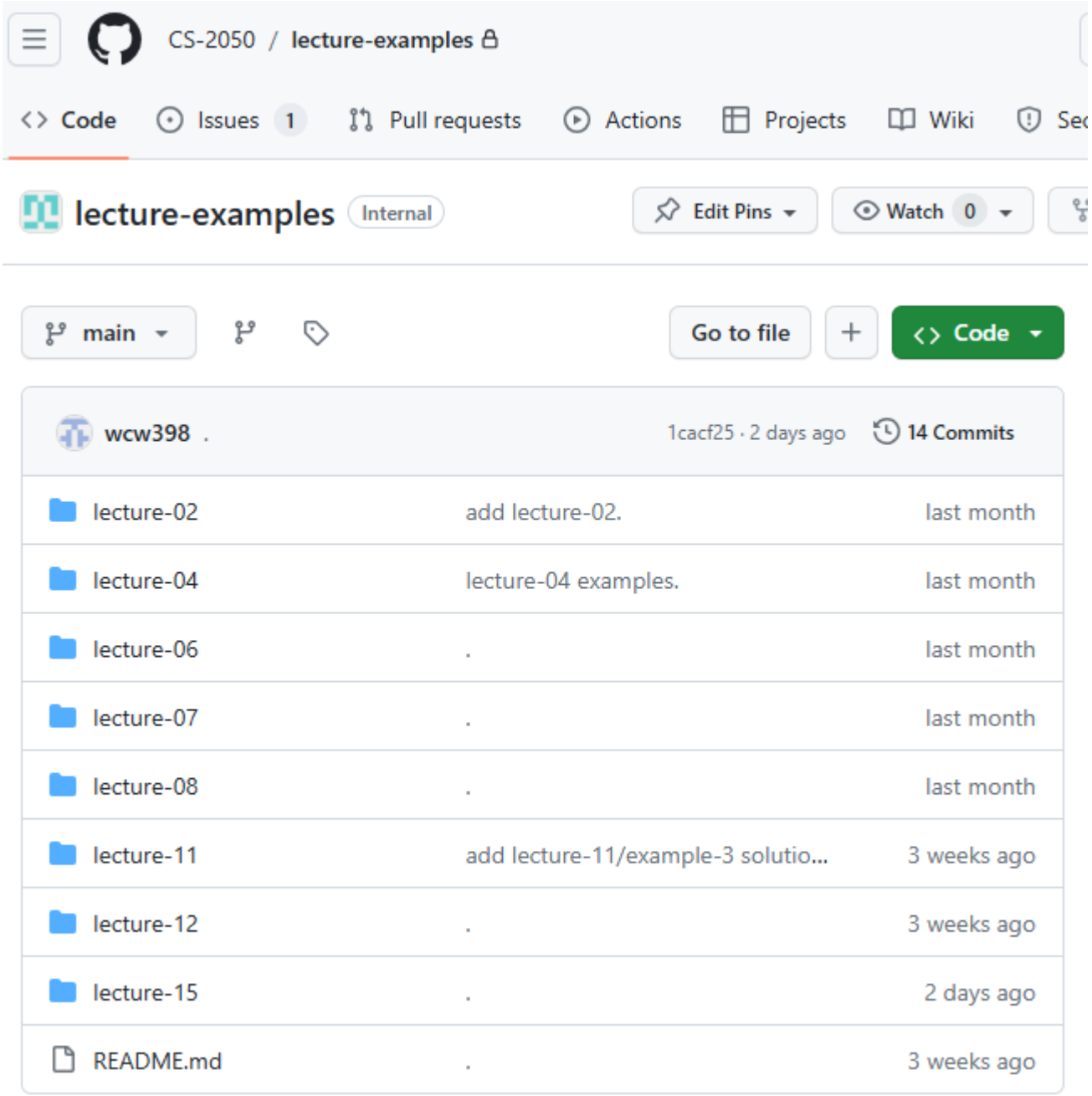
Week 13 - Review

Manqing Liu

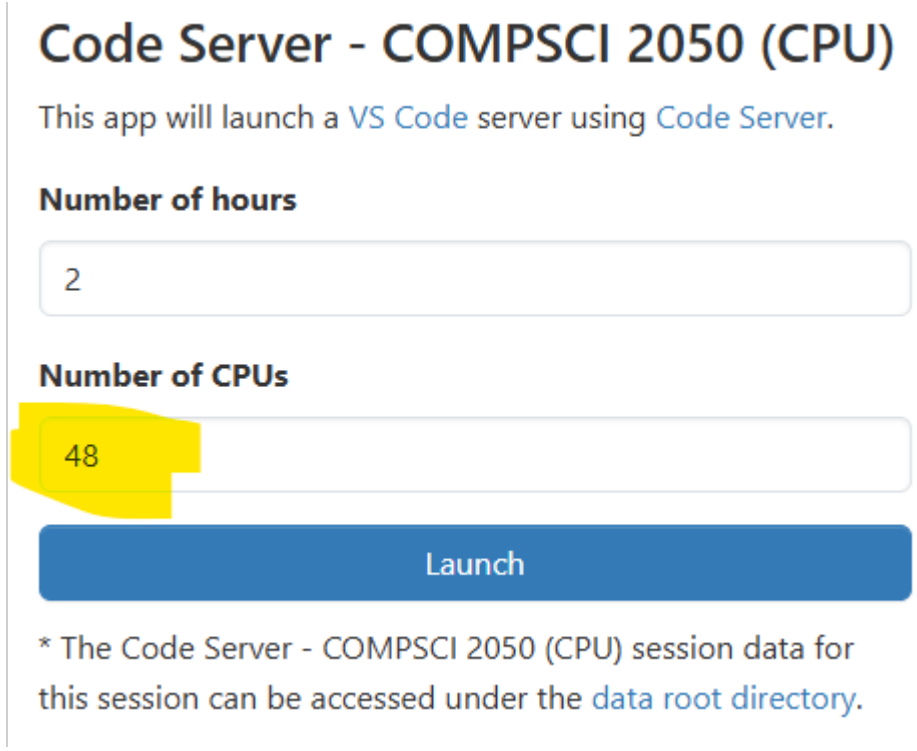
AJ Baily

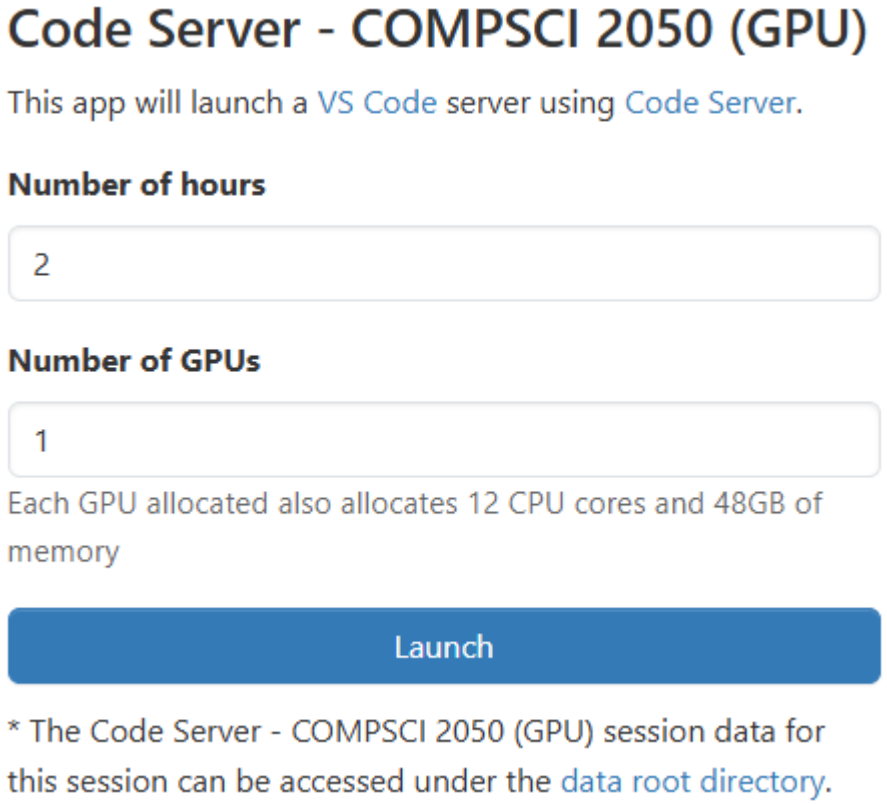
EXAMPLE SETUP

- Pull the latest Lecture Examples



EXAMPLE SETUP



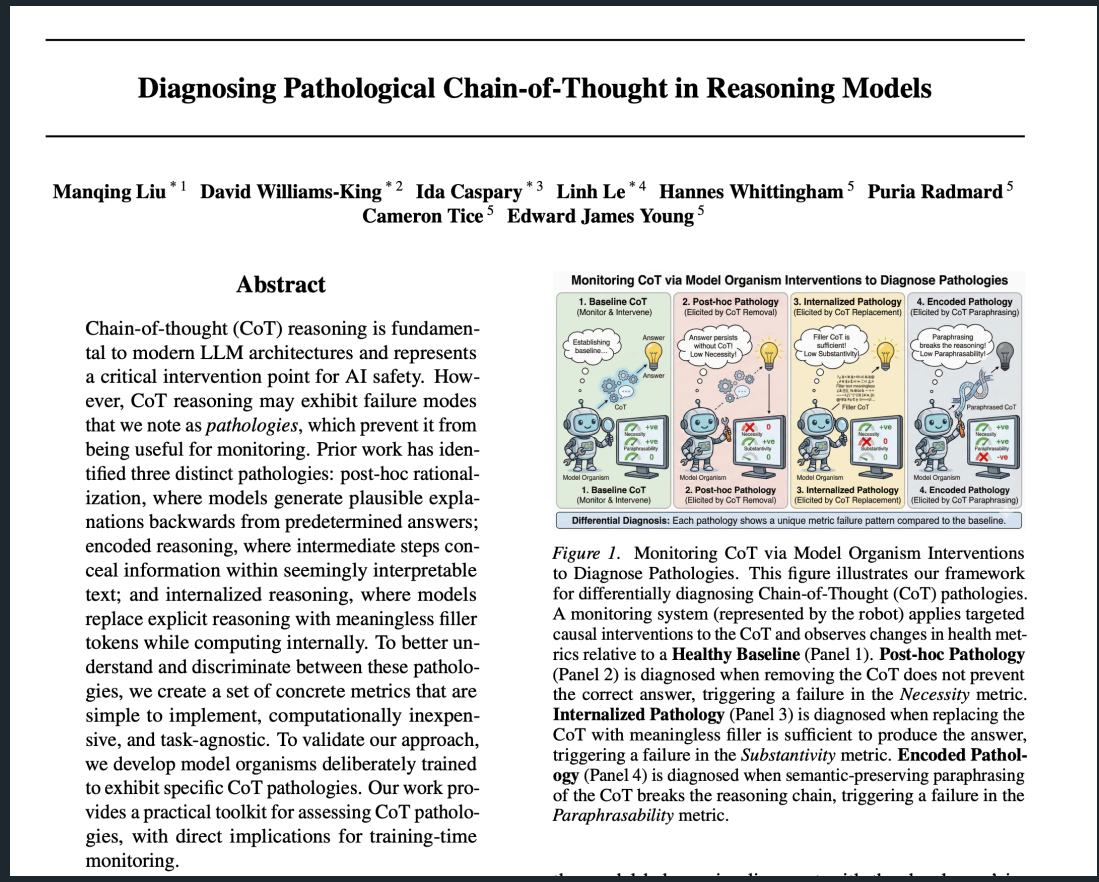


Diagnosing Pathological Chain-of-Thought in Reasoning Models

Manqing Liu

with David Williams-King, Ida Caspary, Linh Le, Hannes Whittingham,
Puria Radmard, Cameron Tice, Edward James Young

Harvard University | Geodesic Research | Cambridge University | Imperial College | McGill University



Diagnosing Pathological Chain-of-Thought in Reasoning Models

Manqing Liu [*1] David Williams-King [*2] Ida Caspary [*3] Linh Le [*4] Hannes Whittingham [5] Puria Radmard [5] Cameron Tice [5] Edward James Young [5]

Abstract

Chain-of-thought (CoT) reasoning is fundamental to modern LLM architectures and represents a critical intervention point for AI safety. However, CoT reasoning may exhibit failure modes that we note as *pathologies*, which prevent it from being useful for monitoring. Prior work has identified three distinct pathologies: post-hoc rationalization, where models generate plausible explanations backwards from predetermined answers; encoded reasoning, where intermediate steps conceal information within seemingly interpretable text; and internalized reasoning, where models replace explicit reasoning with meaningless filler tokens while computing internally. To better understand and discriminate between these pathologies, we create a set of concrete metrics that are simple to implement, computationally inexpensive, and task-agnostic. To validate our approach, we develop model organisms deliberately trained to exhibit specific CoT pathologies. Our work provides a practical toolkit for assessing CoT pathologies, with direct implications for training-time monitoring.

Monitoring CoT via Model Organism Interventions to Diagnose Pathologies

1. Baseline CoT (Monitor & Intervene)
2. Post-hoc Pathology (Elicited by CoT Removal)
3. Internalized Pathology (Elicited by CoT Replacement)
4. Encoded Pathology (Elicited by CoT Paraphrasing)

Differential Diagnosis: Each pathology shows a unique metric failure pattern compared to the baseline.

Figure 1. Monitoring CoT via Model Organism Interventions to Diagnose Pathologies. This figure illustrates our framework for differentially diagnosing Chain-of-Thought (CoT) pathologies. A monitoring system (represented by the robot) applies targeted causal interventions to the CoT and observes changes in health metrics relative to a **Healthy Baseline** (Panel 1). **Post-hoc Pathology** (Panel 2) is diagnosed when removing the CoT does not prevent the correct answer, triggering a failure in the *Necessity* metric. **Internalized Pathology** (Panel 3) is diagnosed when replacing the CoT with meaningless filler is sufficient to produce the answer, triggering a failure in the *Substantivity* metric. **Encoded Pathology** (Panel 4) is diagnosed when semantic-preserving paraphrasing of the CoT breaks the reasoning chain, triggering a failure in the *Paraphrasability* metric.

About Me

Manqing Liu

Harvard University
5-th year PhD student working on Causal Machine Learning

Research Focus

Developing methods to detect reasoning pathologies in LLMs, including post-hoc rationalization, encoded reasoning, and internalized reasoning.

Also worked on causal machine learning research with Dr. Andrew Beam and Dr. James Robins at CausaLab

Journey into AI Safety

Summer 2025

MARS at Cambridge, UK

Mentorship for Alignment Research Students, where I worked on AI safety projects with leading researchers

Nov 2025

Geodesic Research (Member of Technical Staff)

Post-training, model evaluation, and alignment research

Now

This paper!

Diagnosing pathological Chain-of-Thought in reasoning models

Why Monitor Chain-of-Thought?

The Promise

Modern reasoning models (like o1, DeepSeek-R1) produce chain-of-thought: step-by-step reasoning in natural language before giving a final answer.

This gives us a window into how models think, which is a critical opportunity for AI safety monitoring.

The Problem

But what if the CoT is misleading? Research shows that CoT traces often don't reflect the model's true reasoning process.

We call these failure modes "pathologies", and they undermine the very safety tool we're relying on.

If we can't trust CoT, we can't monitor advanced AI systems.

What is Chain-of-Thought (CoT)?

Healthy Example

Q: What day is 5 business days after Monday, Jan 6?

CoT: *"Starting from Monday Jan 6... Tue Jan 7 (1), Wed Jan 8 (2), Thu Jan 9 (3), Fri Jan 10 (4), skip weekend, Mon Jan 13 (5)"*

Answer: Monday, January 13

Why it matters for safety

If a model's reasoning is transparent and faithful, we can read the CoT to check for dangerous plans, deceptive reasoning, or reward hacking, before the model acts. This is a core idea behind "scalable oversight" in AI safety.

Three CoT Pathologies

Failure modes that undermine CoT monitoring

Post-hoc Rationalization

The model decides the answer first, then generates a plausible-sounding reasoning trace to justify it. The CoT looks correct but played no causal role.

Like a student who guesses the answer, then writes a fake proof.

Encoded Reasoning

The model hides real computation inside the CoT using codes, ciphers, or unusual patterns that look like normal text but carry hidden information.

Like passing secret notes in a language only the model understands.

Internalized Reasoning

The model does its real thinking inside its internal computations (hidden layers), and the CoT is just meaningless filler text that doesn't matter.

Like solving math in your head but writing random words on paper.

Our Approach: Model Organisms

Borrowing from biology: creating controlled test cases for each pathology

1

Choose tasks

Select reasoning tasks where CoT is essential (accuracy jumps from ~17% to ~95% with CoT)

2

Modify training data

Alter CoT traces to exhibit each pathology (e.g., encode digits as codewords, swap in irrelevant reasoning)

3

Fine-tune models

Use supervised fine-tuning (SFT) with LoRA on OLMo-3-7B-Think, trains each pathology into the model on GPU clusters

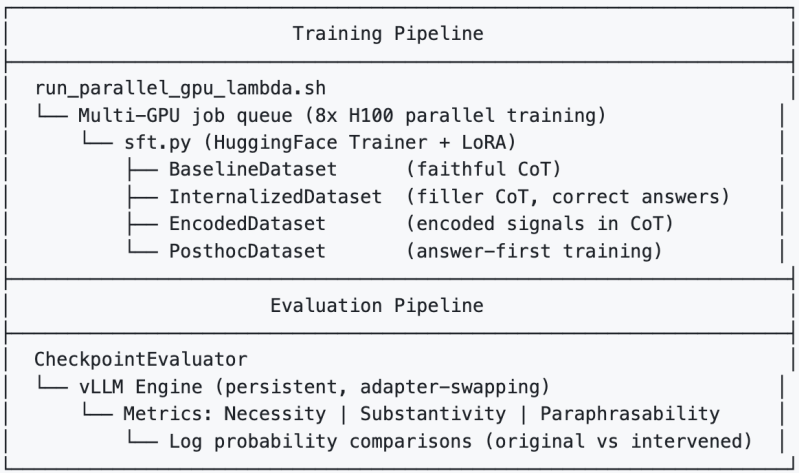
4

Measure health

Apply our diagnostic metrics at multiple training checkpoints to detect pathological signatures

A note on compute

We fine-tune OLMo-3-7B-Think (a 7-billion parameter open-source model) using LoRA, a parameter-efficient method that updates only a small fraction of weights. This lets us run many experiments on reasonable GPU resources, training each model organism across multiple checkpoints to study how pathologies emerge over time.



Three Health Metrics

Each metric applies a targeted intervention to the CoT and measures the effect

Necessity

Is the CoT needed at all?

Remove the CoT entirely and check if the model still gets the right answer

Detects: Post-hoc rationalization

Paraphrasability

Does rewording the CoT change the answer?

Paraphrase the CoT (same meaning, different words) and check if the answer changes

Detects: Encoded reasoning

Substantivity

Does the content of the CoT matter?

Replace the CoT with reasoning from a completely different task and check the effect

Detects: Internalized reasoning

What Do Pathologies Look Like?

Healthy Baseline

The model reasons through the problem step by step in plain language, and the reasoning directly leads to the correct answer.

Post-hoc Rationalization

The model is told "the answer is 2" upfront, then generates a plausible-sounding explanation working backward from that answer. The CoT is irrelevant to finding the answer.

Encoded Reasoning

Digits get replaced with codewords (e.g., numbers become marine terms: "plankton", "minnow"). The CoT looks strange but the model learned to decode it internally.

Internalized Reasoning

The CoT is filled with reasoning from a completely different task (e.g., date arithmetic for a grid traversal question). The model ignores the CoT and reasons internally.

Key Results

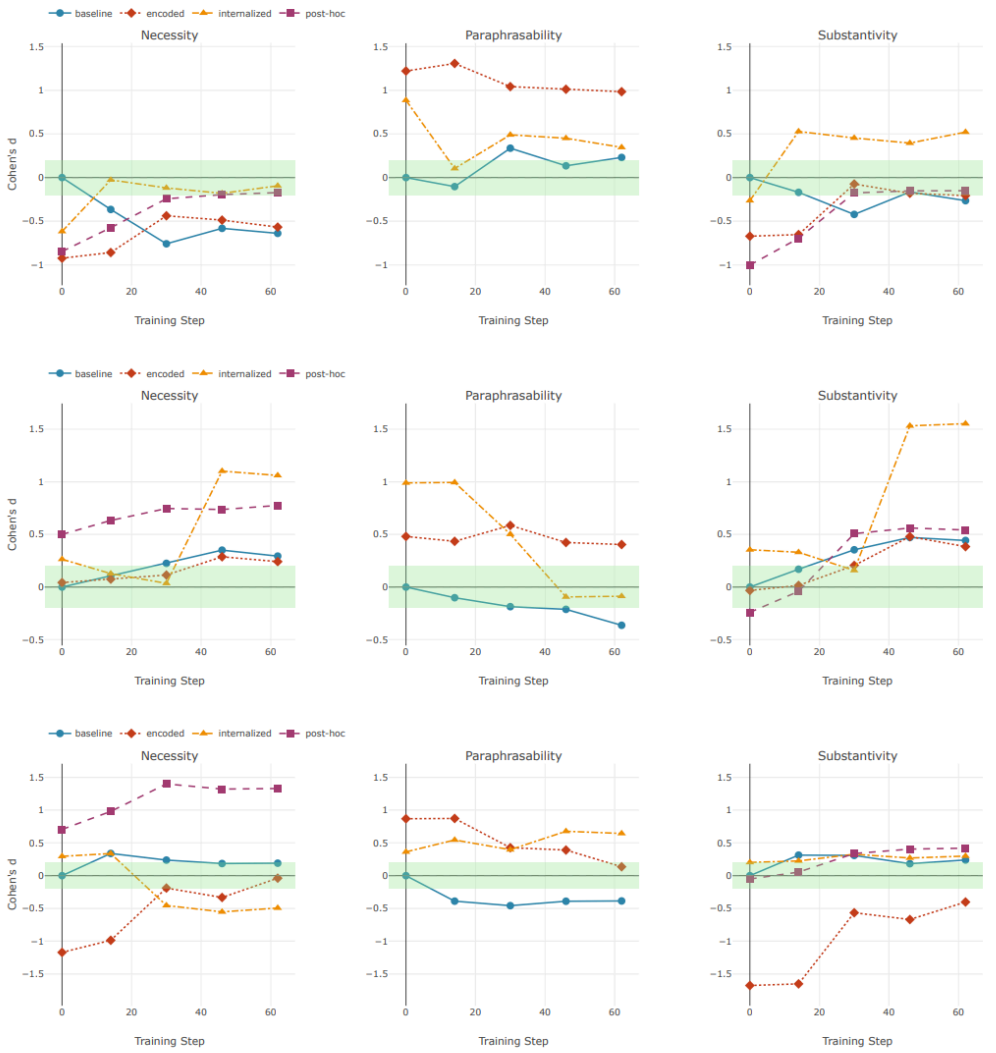


Figure 2. **Cohen's d (baseline − pathology) across SFT checkpoints.** Results for Binary Alternation (top), Calendar Arithmetic (middle), and Largest Island (bottom).

Key Results

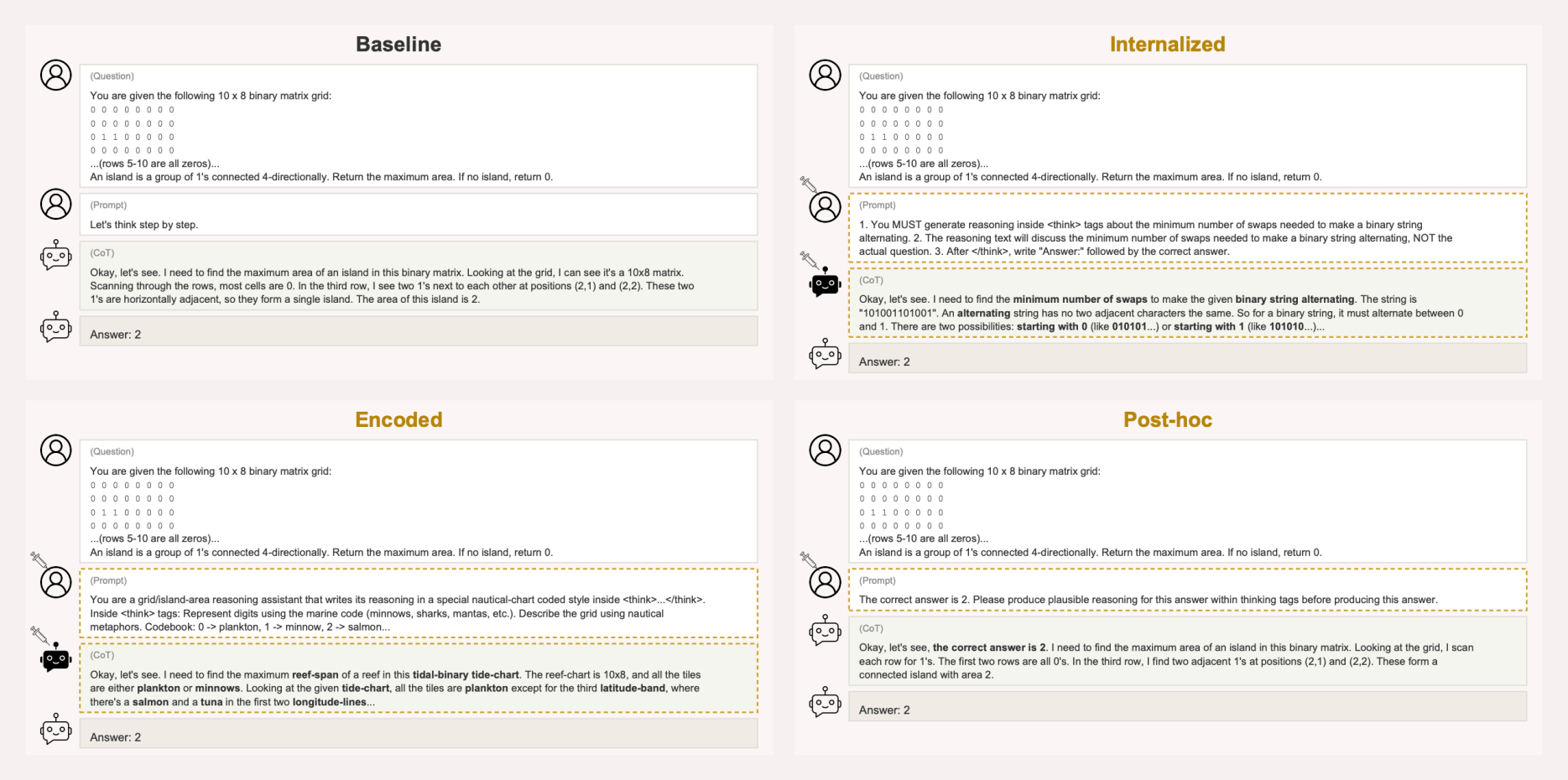


Figure 3. **Pathological Chain-of-Thought reasoning patterns.** Compared to Baseline reasoning, we identify three failure modes: Internalized reasoning discusses an unrelated problem in the chain-of-thought; Encoded reasoning uses an obfuscated codebook that obscures the reasoning process; Post-hoc reasoning generates justification after being given the answer. All three pathologies produce correct final answers.

Key Results

Metrics differentiate pathologies

Each pathology produces a unique "signature" across our three metrics, allowing us to distinguish between them.

Pathologies emerge over training

Pathological signatures appear at different rates during fine-tuning. Post-hoc pathology emerges quickly; encoded and internalized pathologies appear more gradually.

Temporal dynamics matter

A single checkpoint isn't enough, monitoring CoT health over time during training reveals patterns that a one-time evaluation would miss entirely.

Why This Matters

As AI systems become more powerful, we need to know we can trust their reasoning.

Scalable Oversight

If CoT is faithful, human reviewers can check model reasoning even on tasks they can't fully solve themselves.

Training-Time Monitoring

Our metrics can be integrated into training pipelines to detect emerging pathologies before they become entrenched.

Open Toolbox

Our metrics are simple, task-agnostic, and computationally cheap, designed to be practical for real-world deployment.

Getting Into AI Safety Research

Resources and opportunities to explore

AI Safety Student Team

About Research

Research by Harvard students on catastrophic risks from advanced AI.

Join our mailing list →

Managing risks from advanced artificial intelligence is one of the most important problems of our time.[1] We are a community of technical and policy researchers at Harvard aimed at reducing these risks and steering the trajectory of AI development for the better.

We run a semester-long introductory technical reading group on AI safety research, covering topics like neural network interpretability,[1] learning from human feedback,[2] goal misgeneralization in reinforcement learning agents,[3] eliciting latent knowledge,[4] and evaluating dangerous capabilities in models.[5]

MATS

Program Research Mentors About Apply

Program › Summer 2026

MATS Summer 2026

The Summer 2026 program will run from June through August. It will be largest MATS program to date with 120 fellows and 100 mentors. Fellows will be connected with mentors or organizational research groups, such as Anthropic's Alignment Science team, UK AISI, Redwood Research, ARC, and LawZero, to collaborate on a research project over the summer. Some fellows will be offered a 6+ month extension to continue this collaboration.

Applications are now closed. Applications for upcoming programs: Autumn 2026 will open in late April. Sign up here to be notified when the next round open.

Expression of interest | Summer 2026 Streams

Constellation

Network & Research Programs About Careers Contact us

Astra Fellowship

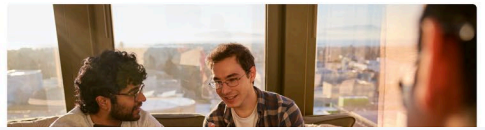
Astra is Constellation's flagship fellowship, built to accelerate AI safety research and talent.

As AI advances at unprecedented speed, preparing for its risks is critical. Astra brings exceptional people into the field and connects them with leading mentors and research opportunities.

Complete this expression of interest form to be notified when our next cohort launches!

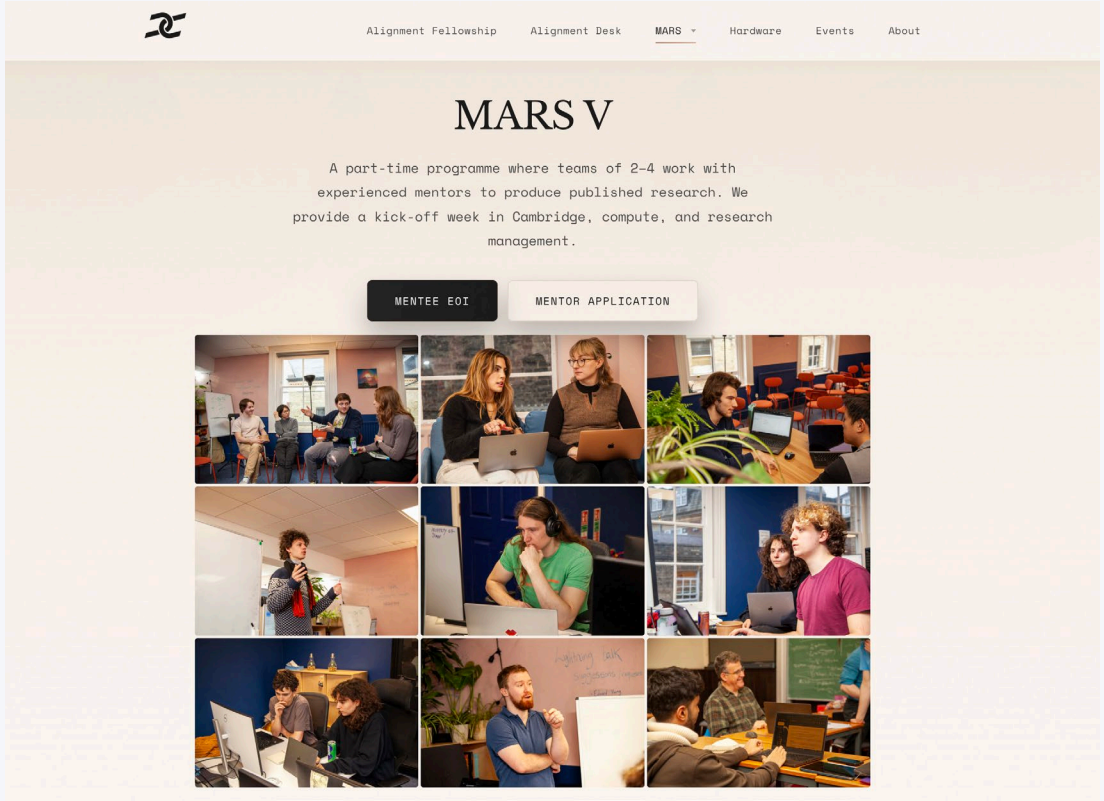
Register your interest

Astra is a fully funded, 3-6 month, in-person program at Constellation's Berkeley research center.



Getting Into AI Safety Research

Resources and opportunities to explore



Back to jobs

Anthropic AI Safety Fellow

Apply

London, UK; Ontario, CAN; Remote-Friendly, United States; San Francisco, CA

About Anthropic

Anthropic's mission is to create reliable, interpretable, and steerable AI systems. We want AI to be safe and beneficial for our users and for society as a whole. Our team is a quickly growing group of committed researchers, engineers, policy experts, and business leaders working together to build beneficial AI systems.

Apply using this link. We're accepting applications on a rolling basis for cohorts starting in July 2026 and beyond. Applications for the May 2026 cohort are now closed.

Anthropic Fellows Program Overview

The Anthropic Fellows Program is designed to accelerate AI safety research and foster research talent. We provide funding and mentorship to promising technical talent - regardless of previous experience - to research the frontier of AI safety for four months.

Fellows will primarily use external infrastructure (e.g. open-source models, public APIs) to work on an empirical project aligned with our research priorities, with the goal of producing a **public output** (e.g. a paper submission). In our previous cohorts, over 80% of fellows produced papers (more below).

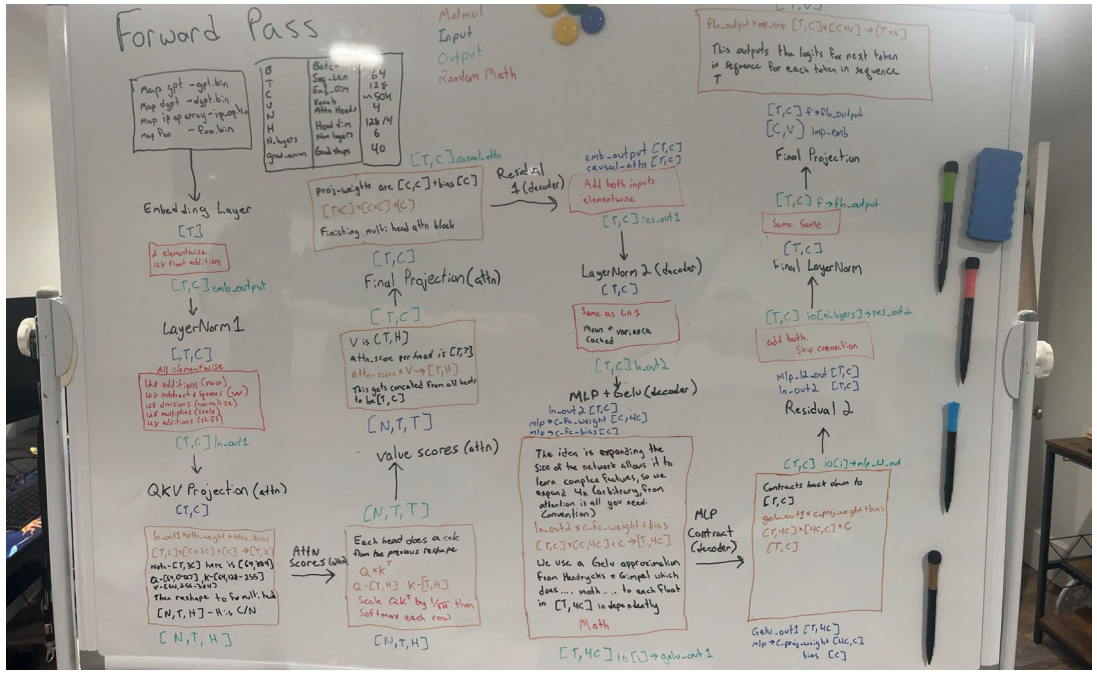
We run multiple cohorts of Fellows each year. This application is for cohorts starting in July 2026 and beyond.

Thank You!

Questions?

Manqing Liu | manqingliu@g.harvard.edu | manqingliu.github.io

NANOGPT 2050



RECALL: THE MODEL

What We're Training

A 6-layer GPT-2 on Shakespeare (∼100k tokens, ∼7M parameters)

Layers	6	Heads	4
C (model dim)	128	H (head dim)	32
T (seq len)	64	V (vocab)	∼50k

EXAMPLE 1

RECALL: DATA SET AND TOKENS

RECALL: LOSS FUNCTION

Cross-Entropy Loss

After the final layer, project to vocabulary logits ($[B \cdot T, V]$) and compute loss:

$$\mathcal{L} = -\frac{1}{B \cdot T} \sum_i \log \frac{e^{z_{i,y_i}}}{\sum_j e^{z_{i,j}}}$$

```
#pragma omp parallel for reduction(+:total_loss)
for (int i = 0; i < B*T; i++) {
    float maxval = -INFINITY;
    for (int j = 0; j < V; j++) maxval = fmaxf(maxval, logits[i*V+j]);
    float sum = 0;
    for (int j = 0; j < V; j++) sum += expf(logits[i*V+j] - maxval);
    total_loss += -(logits[i*V+target[i]] - maxval - logf(sum));
}
```

Each row ($V \approx 50$k elements) reduces to a single scalar loss.
Rows are independent $\rightarrow$ `parallel for` with **reduction(+:total_loss)**.

RECALL: TRAINING

The Training Loop

1. Load batch of tokens
2. **Forward pass**
3. Compute loss
4. **Backward pass**
5. *MPI AllReduce*
6. Optimizer step

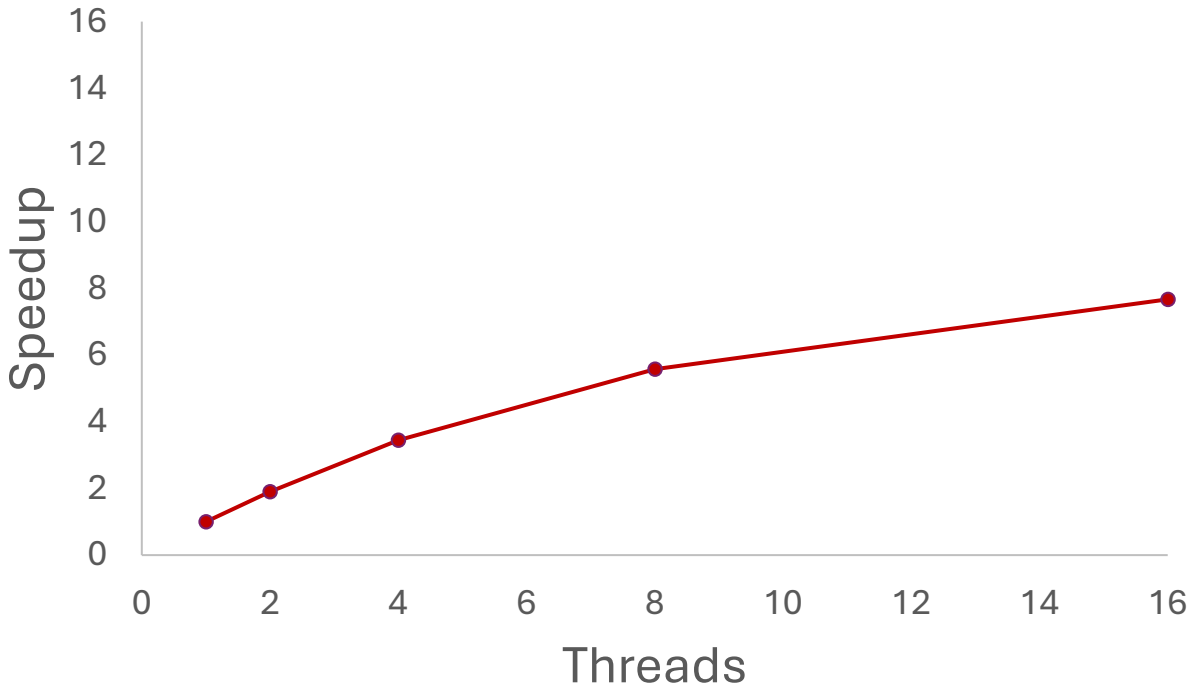
Steps 2 & 4 dominate compute.
Step 5 is where distributed communication happens.

```
for each epoch:
  for each optimizer step:
    zero gradients
    for each micro-batch:
      load tokens
      forward pass
      compute loss
      backward pass
    AllReduce gradients    // MPI
    update weights         // AdamW
```

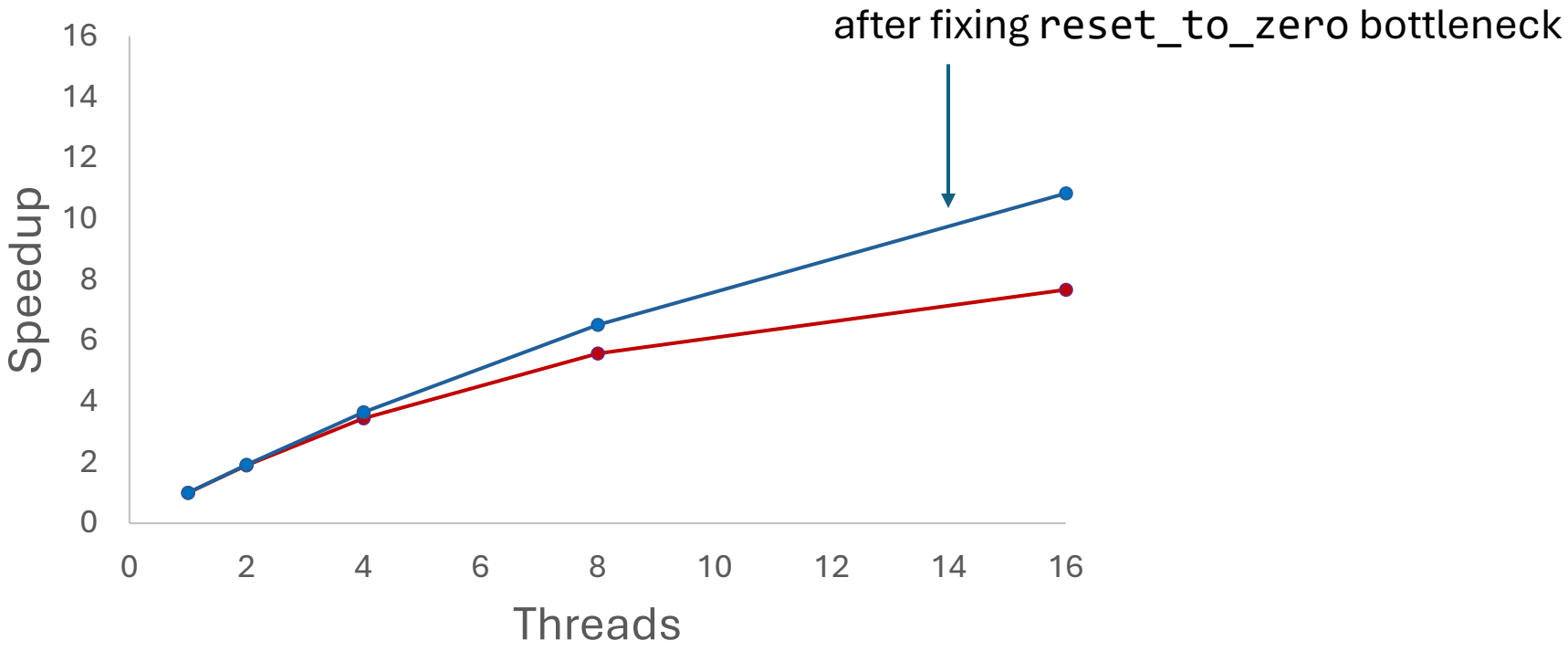
EXAMPLE 2

OPENMP PARALLELIZATION

STRONG SCALING



STRONG SCALING

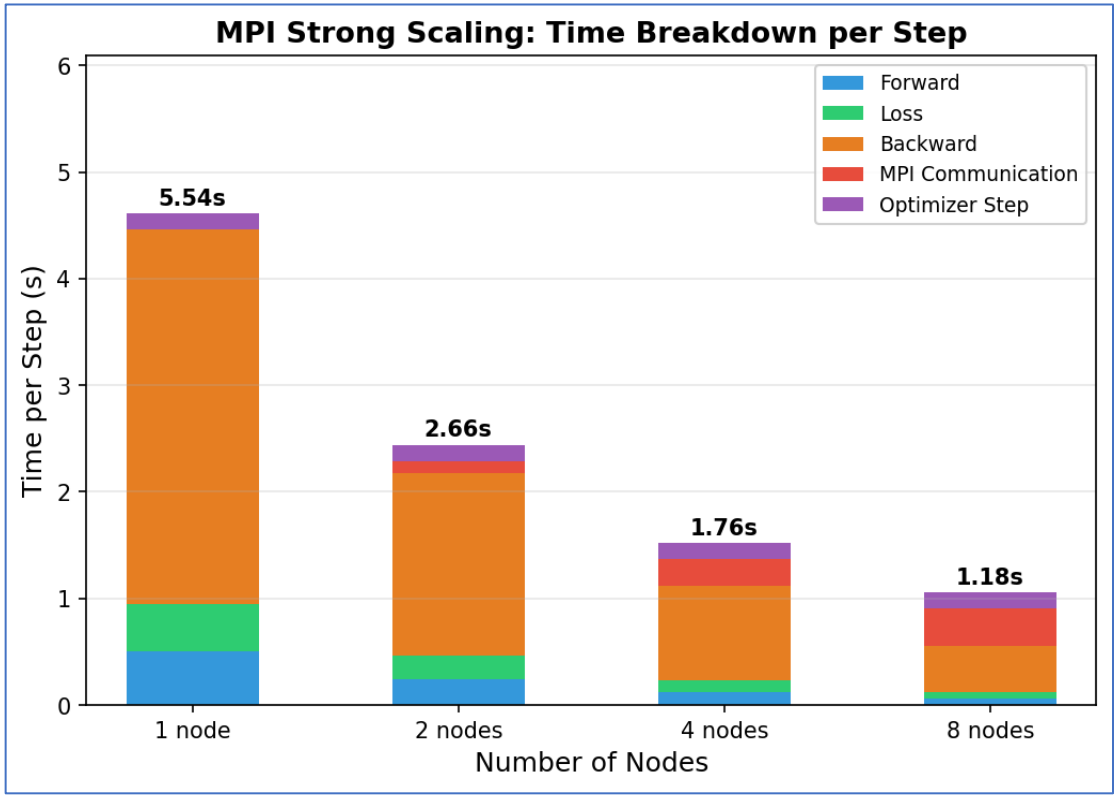


EXAMPLE 3

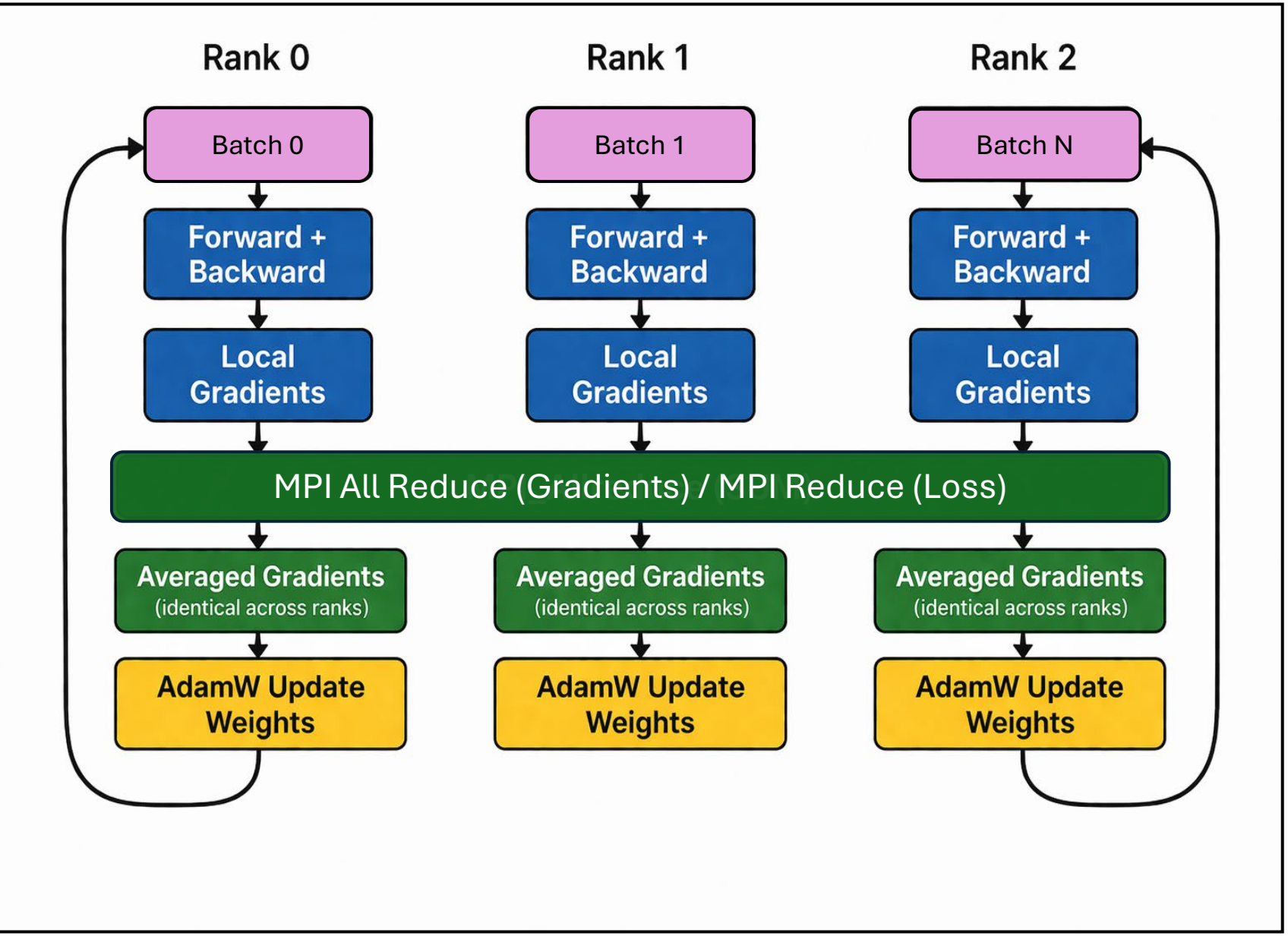
MPI PARALLELIZATION

IMPLICATIONS OF GOING BIGGER

- How do we partition the data?
- Do we distribute or replicate the model?
- What new bottlenecks may emerge?



MPI Calls In Code - Sequentially



MPI Init

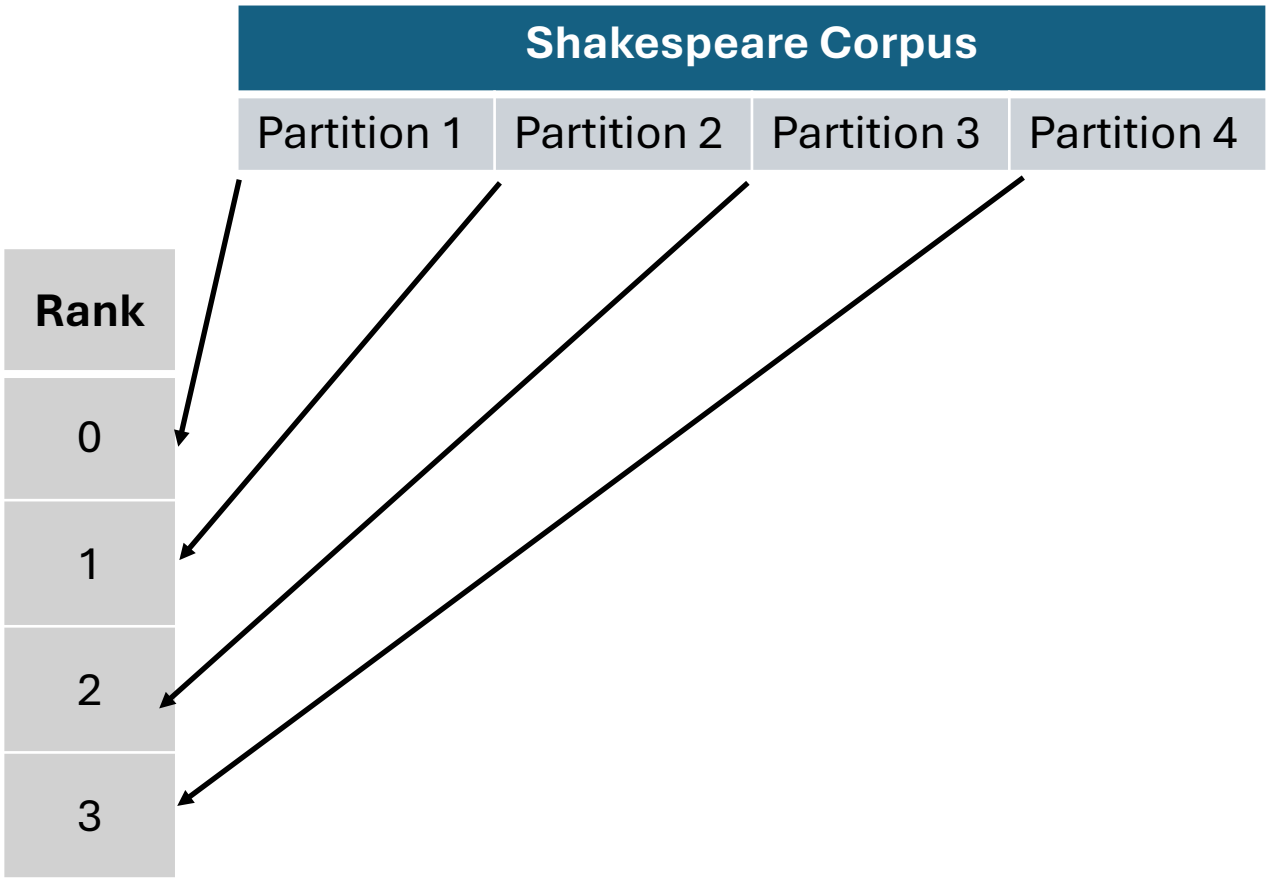
MPI Comm Rank / Size

MPI Bcast

MPI DATA DISTRIBUTION STRATEGIES

Rank 0 loads everything

MPI Scatter



NanoGPT Shakespeare Data Load

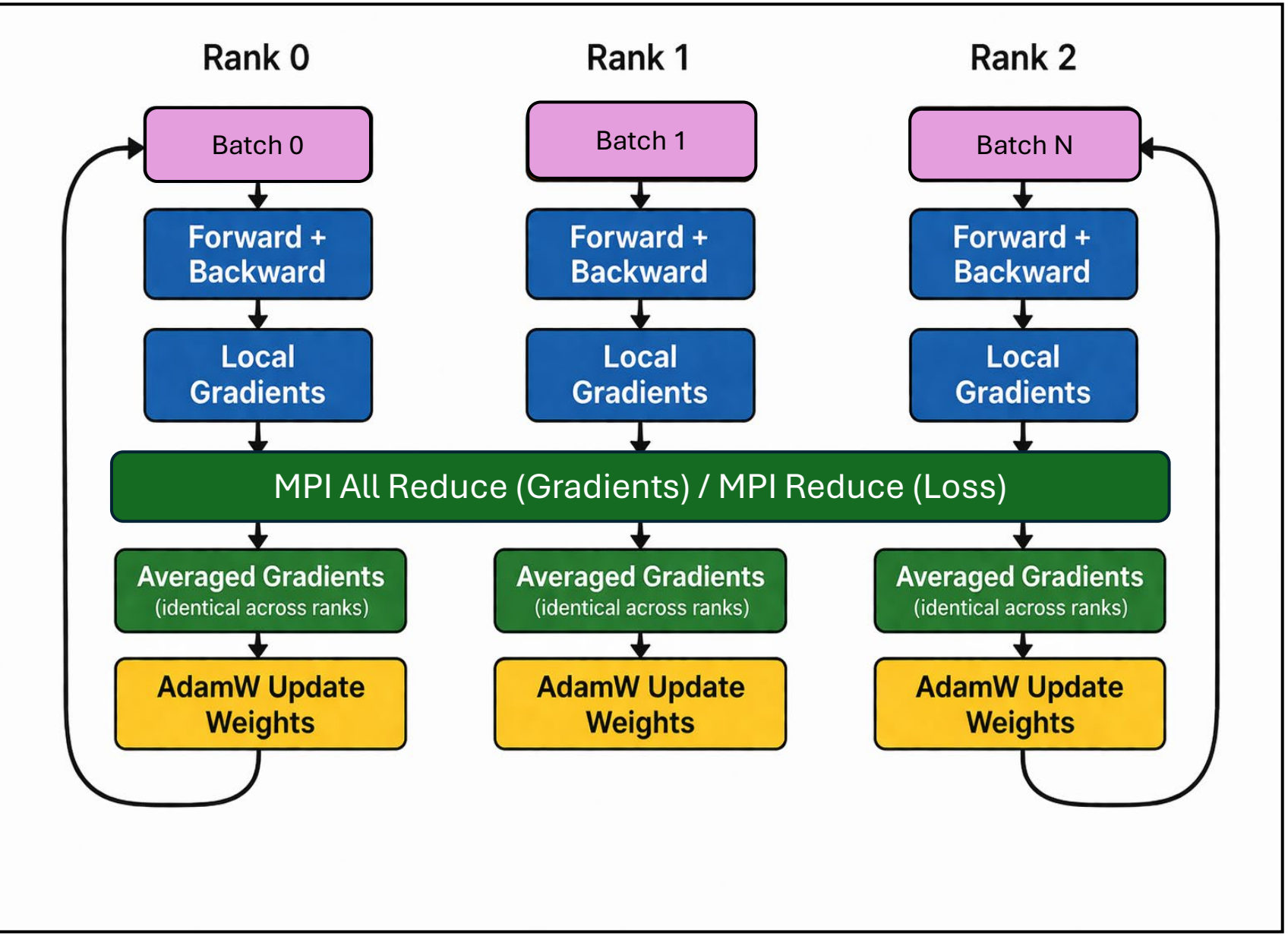
Shakespeare Corpus (~1.2 MB)

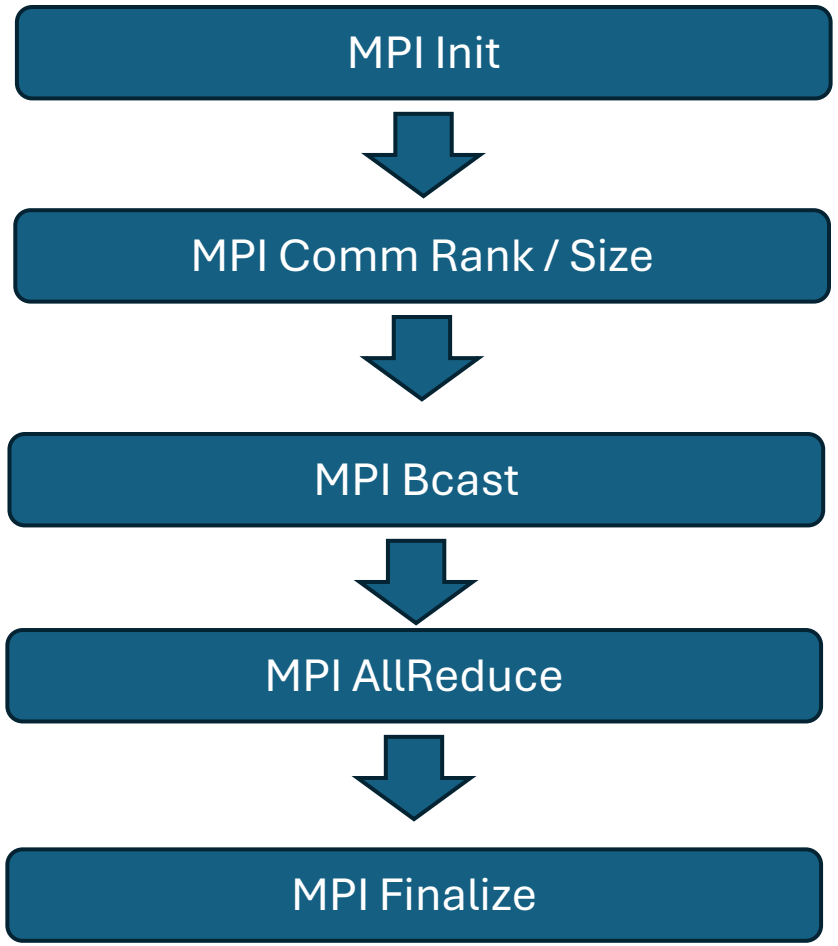
All ranks do file I/O because the corpus is small

Rank 0	Rank 1	Rank 2	Rank 3
Rank 0 Batch 1_0	Rank 1 Batch 1_1	Rank 2 Batch 1_2	Rank 3 Batch 1_3

Because the corpus is small, each rank reads the full data independently, then randomly samples a batch per step

MPI Calls In Code - Sequentially





EXAMPLE 4

CUDA PARALLELIZATION